CO4 AT2

COMPUTER ARCHITECTURE

Q1: 1.Compare L1, L2, and L3 cache levels in terms of latency, bandwidth, and throughput, and analyze their impact on processor performance.

META AI:

1. Latency: L1 ~1-2 cycles (fastest), L2 ~8-12 cycles, L3 ~30-50 cycles. L1 is checked first, miss penalty grows sharply with each level.2. Size & Location: L1 smallest (3264KB) per-core, L2 mid (256KB-1MB) per-core, L3 largest (8-32MB) shared across cores.3. Bandwidth: L1 has highest bandwidth (~1-2 TB/s), L2 medium, L3 lowest. L1 can feed execution units every cycle.4. Throughput / Hits: L1 hit rate ~90%, L2 catches most L1 misses, L3 catches inter-core sharing. Higher L1/L2 hit rate = fewer stalls.5. Performance Impact: L1 dictates IPC, L2 balances latency vs hit rate, L3 reduces DRAM access (~200+ cycles). Bigger/faster caches = lower average memory access time, but bigger L1 increases its own latency, so design is a trade-off.

CLAUDE AI:

L1, L2, L3 Cache Comparison:

1. **Latency:** L1 has the lowest latency (~1–4 CPU cycles) since it's built directly into the core with the smallest size (16–64KB). L2 has moderate latency (~10–20 cycles), being slightly farther from the core (256KB–1MB). L3 has the highest latency (~30–50+ cycles) as it's shared across cores and much larger (several MB to tens of MB).
2. **Bandwidth:** L1 offers the highest bandwidth due to its proximity to the core and dedicated per-core access. L2 bandwidth is lower than L1 but still high, also typically per-core. L3 has the lowest bandwidth among the three since it's shared among multiple cores, causing contention.
3. **Throughput:** L1 delivers the highest throughput, handling the most frequent, small, fast accesses per cycle. L2 throughput is moderate, acting as a buffer between fast L1 and slower L3/main memory. L3 has the lowest throughput but compensates with larger capacity to reduce main memory accesses.
4. **Size vs Speed Tradeoff:** As cache level increases (L1→L2→L3), size increases but speed decreases. This hierarchy exists because larger, faster memory close to the core is expensive and physically limited, so designers balance speed and capacity across levels.

5. **Impact on Processor Performance:** A well-designed cache hierarchy reduces average memory access time by exploiting temporal and spatial locality — most accesses hit L1, fewer hit L2, and even fewer fall to L3, drastically cutting the need to access slow main memory (DRAM). Higher cache hit rates directly translate to fewer CPU stall cycles, better IPC (instructions per cycle), and overall higher processor throughput. Poor cache utilization (frequent misses) causes pipeline stalls and degrades performance significantly, regardless of clock speed.

CHATGPT:

1. **L1 Cache:** It is the smallest and fastest cache, with the lowest latency and highest bandwidth. It is located closest to the CPU core and provides very high throughput for frequently used data and instructions.
2. **L2 Cache:** It is larger than L1 but slightly slower, with higher latency and lower bandwidth than L1. It acts as an intermediate cache and reduces the need to access the slower main memory.
3. **L3 Cache:** It is the largest and slowest of the three cache levels, with higher latency but large storage capacity. It is usually shared among multiple CPU cores and provides moderate bandwidth and throughput.
4. **Performance Impact:** Faster cache access reduces CPU waiting time and improves instruction execution speed. L1 provides the fastest access, while L2 and L3 help prevent frequent accesses to RAM.
5. **Overall Comparison:** L1 has the lowest latency and highest throughput, followed by L2 and then L3. A well-designed cache hierarchy improves overall processor performance by reducing memory access delays.

Q1 – L1/L2/L3 Cache Comparison

1. **Claude** – Only one to explicitly flag the L1 size-vs-latency trade-off *and* tie hit rates to IPC/AMAT. Most complete.
2. **Meta AI** – Same trade-off insight, solid numbers, but less structured explanation of *why* it matters for performance.
3. **ChatGPT** – Correct but shallow. No quantitative latency figures, no trade-off discussion, reads like a definition list.

Q2: Differentiate SRAM and DRAM by evaluating their latency, bandwidth, and throughput characteristics in cache and main memory design.

META AI:

SRAM vs DRAM - 5 point differentiation:

1. Cell Structure & Latency: SRAM uses 6 transistors (no refresh) = 1-10ns latency. DRAM uses 1 transistor + capacitor (needs refresh) = 50-100ns latency.

2. Bandwidth & Throughput: SRAM delivers very high bandwidth and can sustain 1 read/write per cycle. DRAM has lower effective bandwidth due to row activation, precharge and refresh overhead.

3. Usage in Hierarchy: SRAM is used for L1, L2, L3 cache because speed matters. DRAM is used for main memory because density and cost matter.

4. Density, Cost, Power: SRAM is low density, expensive, high static power. DRAM is high density, cheap per bit, low static power but needs refresh power.

5. Performance Impact: SRAM reduces Average Memory Access Time by catching hits quickly. DRAM defines system throughput ceiling - cache miss to DRAM stalls CPU for 200+ cycles, so cache size tries to minimize DRAM accesses.

CLAUDE AI:

SRAM vs DRAM Comparison:

1. **Structure & Basis:** SRAM (Static RAM) stores each bit using a 6-transistor flipflop circuit, requiring no refresh. DRAM (Dynamic RAM) stores each bit as charge in a capacitor with 1 transistor, needing constant refresh cycles to prevent data loss.
2. **Latency:** SRAM has very low latency (~1–2ns) since flip-flops switch states instantly without charge/discharge delays. DRAM has significantly higher latency (~50–100ns) due to capacitor charging time and mandatory refresh operations that periodically block access.
3. **Bandwidth:** SRAM provides higher bandwidth per cell since it can be accessed continuously without refresh interruptions. DRAM has lower effective bandwidth because refresh cycles and row/column addressing (multiplexed address bus) introduce overhead and periodic unavailability.

4. **Throughput:** SRAM achieves higher throughput, making it ideal for cache where speed is critical despite higher cost. DRAM has lower throughput per access but compensates through burst-mode transfers and higher density, making it more throughput-efficient for bulk sequential data in main memory.
5. **Design Role & Tradeoff:** SRAM is used for cache (L1/L2/L3) because it's fast but expensive and less dense (fewer bits per chip area). DRAM is used for main memory because it's cheaper, denser (more bits per chip area), and sufficient for bulk storage — the classic speed-vs-cost-vs-density tradeoff drives this division in memory hierarchy design.

CHATGPT :

1. **SRAM:** SRAM has very low latency because it does not require refreshing. It is mainly used for CPU cache memory where fast access is required.
2. **DRAM:** DRAM has higher latency than SRAM because it requires periodic refreshing. It is mainly used as the computer’s main memory (RAM).
3. **Bandwidth:** SRAM generally provides higher bandwidth and faster data transfer, while DRAM provides comparatively lower bandwidth but supports larger memory capacities.
4. **Throughput:** SRAM offers higher throughput because the CPU can access data more quickly. DRAM has lower throughput due to higher access time and refresh operations.
5. **Overall Design:** SRAM is preferred for **cache memory** because of its speed, while DRAM is preferred for **main memory** because it is cheaper and provides much greater storage capacity.

SRAM vs DRAM

1. **Claude – Explains *why* (6T flip-flop vs 1T+capacitor), links refresh overhead to bandwidth loss, closes with the cost/density/speed trade-off explicitly.**
2. **Meta AI – Same substance, correct numbers, more compressed — less explanatory depth.**
3. **ChatGPT – No numbers at all for latency. Purely qualitative, weakest technical grounding.**

Q3: Compare virtual memory with paging and cache memory mechanisms in terms of access latency and system throughput.

META AI:

Virtual Memory (with Paging) vs Cache Memory - 5 points:

1. Purpose: Cache hides DRAM latency to improve speed. Virtual memory with paging hides disk latency and provides isolation, large address space, not speed.

2. Access Latency: Cache: 1-50 cycles (L1 to L3). TLB hit + paging: ~1-2 cycles extra. TLB miss + page walk: ~30-100 cycles. Page fault to disk: ~10 million cycles (10ms).

3. Hit/Miss Handling: Cache miss is handled by hardware fetching from L2/L3/DRAM. Page fault is handled by OS, triggers disk I/O and context switch, very expensive.

4. Throughput Impact: Cache hit increases IPC and core throughput directly. Paging keeps throughput stable via multiprogramming - while one process waits for disk, another runs, but page fault rate too high causes thrashing and throughput collapses.

5. Granularity & Management: Cache: 64B block, fully hardware managed. Virtual memory: 4KB/2MB page, OS + hardware (MMU/TLB) managed. Cache optimizes for temporal/spatial locality, paging optimizes for memory utilization and protection.

CLAUDE AI:

Virtual Memory (Paging) vs Cache Memory Comparison:

1. **Purpose & Mechanism:** Virtual memory with paging maps logical addresses to physical addresses in fixed-size blocks (pages), letting programs use more address space than physically available RAM, with pages swapped between RAM and disk.

Cache memory stores frequently accessed data closer to the CPU to reduce average access time — it doesn't extend address space, it accelerates existing memory access.

2. **Access Latency:** Cache memory has extremely low latency (1–50 cycles depending on level: L1/L2/L3) since it's on-chip SRAM. Virtual memory paging has drastically higher latency when a page fault occurs (accessing disk/SSD), potentially costing millions of cycles — far worse than any cache miss, since disk I/O is orders of magnitude slower than RAM.
3. **Bandwidth & Throughput Impact:** Cache hits sustain high throughput by feeding the CPU pipeline continuously with minimal stalls. Paging, when page faults are frequent (thrashing), severely cripples system throughput because the CPU must wait for slow disk transfers, and the OS spends cycles on page replacement algorithms (LRU, FIFO, etc.) instead of useful computation.
4. **Hit/Miss Penalty Scale:** A cache miss costs going to the next cache level or main memory (nanoseconds). A page fault (paging "miss") costs going to secondary storage (milliseconds) — roughly 100,000x costlier in relative terms, making page fault minimization far more critical to overall system performance than cache miss minimization.
5. **Role in Memory Hierarchy:** Both are hierarchy-management mechanisms working on the locality principle, but at different levels: cache bridges CPU-toRAM speed gap using hardware (transparent, automatic, no OS involvement), while virtual memory paging bridges RAM-to-disk capacity gap using OSmanaged page tables, TLBs, and page replacement policies. Together they form a layered system where latency increases and throughput drops as you move farther from the CPU.

CHATGPT:

1. **Cache Memory:** Cache memory has the lowest access latency because it is located close to the CPU. It improves system throughput by providing frequently used data quickly.
2. **Paging:** Paging divides memory into fixed-size pages and frames. It has higher latency because page table lookups are required, but it allows efficient use of physical memory.
3. **Virtual Memory:** Virtual memory provides a larger logical memory space by using secondary storage. Its access latency is much higher than RAM because disk/SSD access is slower.
4. **System Throughput:** Cache memory greatly increases throughput by reducing CPU waiting time, while paging and virtual memory can reduce throughput when frequent page faults occur.

5. **Overall Comparison:** Cache focuses on **speed**, paging focuses on **efficient memory management**, and virtual memory focuses on **providing more usable memory** than the available physical RAM.

Virtual Memory/Paging vs Cache

1. **Meta AI** – Only answer that separates TLB hit / TLB miss+page walk / page fault into distinct latency tiers. That granularity is what an examiner is actually testing here.
2. **Claude** – Strong on throughput/thrashing and OS-managed replacement (LRU/FIFO), but skips the TLB layer entirely — a real gap.
3. **ChatGPT** – No cycle-level breakdown anywhere, treats "paging" and "virtual memory" as loosely separate without tying latency numbers to either.

Q4: Analyze the differences between NAND Flash and DRAM with respect to latency, bandwidth, and suitability for hierarchical memory systems.

META AI:

NAND Flash vs DRAM - 5 points:

1. Latency: DRAM ~50-100ns random access, symmetric R/W. NAND Flash ~50100us read, 1-5ms write/program, erase is ms to tens of ms. Flash is 1000x slower.

2. Bandwidth & Throughput: DRAM ~20-100 GB/s per channel, high random IOPS. NAND Flash ~500MB/s - 7GB/s sequential via controller, but random write throughput collapses due to garbage collection and write amplification.

3. Endurance & Write Mechanism: DRAM has unlimited endurance and byteaddressable overwrites. NAND has block erase before write, limited P/E cycles (1k100k), needs FTL and wear leveling.

4. Volatility & Cost: DRAM is volatile, expensive, low density. NAND is non-volatile, cheap per GB, high density, retains data without power.

5. Suitability in Hierarchy: DRAM is main memory layer - directly CPU addressable for active working set. NAND is storage layer (SSD) below DRAM - used for paging, checkpointing, storage hierarchy. It cannot replace DRAM due to latency and endurance, but extends hierarchy for capacity.

CLAUDE AI :

NAND Flash vs DRAM Comparison:

1. **Structure & Persistence:** DRAM is volatile — stores data as capacitor charge, loses all data on power loss, requires constant refresh. NAND Flash is nonvolatile — stores data as trapped charge in floating-gate transistors, retains data without power, no refresh needed.
2. **Latency:** DRAM has very low latency (~50–100ns) for both reads and writes. NAND Flash has much higher latency (~10,000–100,000ns / microseconds range), and writes are especially slow because they require erase-before-write at the block level, unlike DRAM's direct byte-level access.
3. **Bandwidth:** DRAM offers significantly higher bandwidth (tens of GB/s per channel) due to direct parallel bus access. NAND Flash has lower bandwidth (though modern NVMe SSDs with multiple channels narrow this gap considerably), limited further by controller overhead and error correction (ECC) processing.
4. **Access Granularity & Endurance:** DRAM allows fine-grained byte-addressable access with effectively unlimited write endurance. NAND Flash operates at page/block granularity (reads by page, erases by block), and has limited write endurance (wears out after a finite number of program/erase cycles), requiring wear-leveling algorithms.
5. **Suitability in Memory Hierarchy:** DRAM sits as main memory — fast enough to feed the CPU directly but too expensive/volatile for bulk storage. NAND Flash sits as secondary/persistent storage (SSDs) — bridges the gap between DRAM and slower HDDs, offering non-volatility and higher density at lower cost per bit, but its latency makes it unsuitable for direct CPU-level operations. It fits between DRAM and traditional magnetic storage in the hierarchy, trading speed for persistence and capacity.

CHATGPT:

1. **Latency:** DRAM has much lower access latency than NAND Flash, making it suitable for fast, direct access by the CPU. NAND Flash has higher latency, especially for write and erase operations.

2. **Bandwidth:** DRAM generally provides higher bandwidth and faster data transfer rates than NAND Flash, allowing it to handle frequent memory accesses efficiently.
3. **Volatility:** DRAM is volatile and loses data when power is removed, while NAND Flash is non-volatile and retains data without power.
4. **Memory Hierarchy:** DRAM is mainly used as **main memory**, while NAND Flash is used for **secondary storage** such as SSDs because of its high capacity and non-volatility.
5. **Overall Suitability:** DRAM is preferred for high-speed processing and system throughput, whereas NAND Flash is preferred for long-term storage. Both can work together in a hierarchical memory system to balance **speed, capacity, and cost**.

NAND Flash vs DRAM

1. **Claude** – Cleanest 5-point split: persistence, latency, bandwidth, granularity/endurance, hierarchy role — no overlap between points.
2. **Meta AI** – Equally technical (write amplification, FTL, wear leveling called out explicitly) but points bleed into each other more.
3. **ChatGPT** – Correct direction, zero numbers, no mention of write amplification, erase-before-write, or wear leveling — missing the mechanisms that actually explain the latency gap.

Q5: Compare MRAM and RRAM technologies by examining their performance in terms of access time, throughput, and energy efficiency.

META AI:

MRAM vs RRAM - 5 points:

1. Mechanism: MRAM (STT-MRAM) stores data via magnetic tunnel junction polarization. RRAM/ReRAM stores via resistive filament formation in oxide.

2. Access Time: MRAM read ~5-20ns, write ~10-50ns near DRAM speed. RRAM read ~20-100ns, write ~100ns-1us, slower and more variable due to set/reset.

3. Throughput & Endurance: MRAM high endurance (>1e12 cycles) supports high random write throughput, suitable for cache/main memory. RRAM endurance lower (∼ 1e6-1e9) and write needs higher voltage pulses, limits sustained throughput.

4. Energy Efficiency: MRAM read energy very low, write energy moderate (needs high current). RRAM write energy high for forming filament, but zero standby leakage. Both are non-volatile so eliminate refresh power vs DRAM/SRAM.

5. Hierarchy Suitability: MRAM candidate for L3/L4 cache and persistent main memory replacement due to speed and endurance. RRAM candidate for storage-class memory and embedded NVM replacing NAND due to density and lower cost, not for L1/L2.

CLAUDE AI :

MRAM vs RRAM Comparison:

1. **Storage Mechanism:** MRAM (Magnetoresistive RAM) stores data using magnetic tunnel junctions (MTJ), where resistance is determined by the relative magnetic orientation of two ferromagnetic layers. RRAM (Resistive RAM) stores data by changing resistance in a dielectric material through formation/rupture of conductive filaments.
2. **Access Time:** MRAM has very fast access time (~2–20ns), comparable to SRAM/DRAM, since switching magnetic states is quick and doesn't involve chemical/physical filament changes. RRAM has slightly higher access time (~10–50ns) since filament formation/rupture is a physical process that takes longer than magnetic switching, though still much faster than NAND Flash.
3. **Throughput:** MRAM delivers higher and more consistent throughput due to its deterministic switching mechanism and symmetric read/write speeds. RRAM throughput is slightly lower and can be less consistent due to filament variability (device-to-device and cycle-to-cycle variation), though still significantly better than NAND Flash.
4. **Energy Efficiency:** RRAM is generally more energy-efficient for write operations since it requires lower switching current/voltage. MRAM (especially STT-MRAM) requires relatively higher write current for reliable switching, though it's still far more efficient than DRAM refresh cycles or Flash erase operations.
5. **Suitability & Endurance:** MRAM offers superior endurance (10^12–10^15 cycles) making it suitable as a potential universal memory replacing both SRAM and DRAM in future hierarchies. RRAM has good but lower endurance (10^6– 10^12 cycles) and

better scalability to smaller technology nodes, positioning it as a strong candidate for high-density non-volatile cache or storage-class memory, though with more variability challenges to solve.

CHATGPT :

1. **Access Time:** MRAM provides fast access with low read/write latency, while RRAM also offers fast access but its latency can vary depending on the device design.
2. **Throughput:** MRAM can provide high read and write throughput and is suitable for applications requiring frequent memory operations. RRAM also offers good throughput with potential for high-density memory.
3. **Energy Efficiency:** MRAM is energy-efficient because it retains data without continuous power. RRAM can achieve very low energy consumption, especially for write operations.
4. **Memory Characteristics:** Both MRAM and RRAM are **non-volatile memories**, meaning they retain data even when power is removed. This makes them useful as alternatives to traditional volatile memories.
5. **Overall Comparison:** MRAM is preferred for **fast access and high endurance**, while RRAM is attractive for **high density and low-power applications**. Both technologies can improve energy efficiency in hierarchical memory systems.

MRAM vs RRAM

1. **Claude** – Explains the physical mechanism first (MTJ vs filament) before quantifying, which is why the access-time numbers make sense instead of just being asserted.
2. **Meta AI** – Comparable numbers and endurance ranges, but energy efficiency point is thinner (doesn't explain why RRAM write energy is high despite being efficient elsewhere).
3. **ChatGPT** – No access-time numbers, no endurance cycle counts, no mechanism explanation. Weakest of the three by a wide margin here.

Overall pattern: ChatGPT is 3rd on every single question — it never gives quantitative figures or explains underlying mechanisms, just restates definitions. Claude wins 4/5 on structure + mechanism explanation + correct numbers. Meta AI wins Q3 specifically because it's the only one that modeled the TLB layer correctly — worth noting since that's an easy point to lose on this topic.